JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class OutputStream

java.lang.Object
    java.io.OutputStream

All Implemented Interfaces:

`Closeable`, `Flushable`, `AutoCloseable`

Direct Known Subclasses:

`ByteArrayOutputStream`, `FileOutputStream`, `FilterOutputStream`, `ObjectOutputStream`, `OutputStream`, `PipedOutputStream`

```
public abstract class OutputStream
extends Object
implements Closeable, Flushable
```

This abstract class is the superclass of all classes representing an output stream of bytes. An output stream accepts output bytes and sends them to some sink.

Applications that need to define a subclass of `OutputStream` must always provide at least a method that writes one byte of output.

Since:

`JDK1.0`

See Also:

`BufferedOutputStream`, `ByteArrayOutputStream`, `DataOutputStream`, `FilterOutputStream`, `InputStream`, `write(int)`

Constructor Summary

Constructors

Constructor	Description

OutputStream()

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
void	close()	Closes this output stream and releases any system resources associated with this stream.
void	flush()	Flushes this output stream and forces any buffered output bytes to be written out.
void	write(byte[] b)	Writes b.length bytes from the specified byte array to this output stream.
void	write(byte[] b, int off, int len)	Writes len bytes from the specified byte array starting at offset off to this output stream.
abstract void	write(int b)	Writes the specified byte to this output stream.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

OutputStream

```
public OutputStream()
```

Method Detail

write

```
public abstract void write(int b)
                    throws IOException
```

Writes the specified byte to this output stream. The general contract for `write` is that one byte is written to the output stream. The byte to be written is the eight low-order bits of the argument `b`. The 24 high-order bits of `b` are ignored.

Subclasses of `OutputStream` must provide an implementation for this method.

Parameters:

`b - the byte.`

Throws:

`IOException - if an I/O error occurs. In particular, an IOException may be thrown if the output stream has been closed.`

write

```
public void write(byte[] b)
           throws IOException
```

Writes `b.length` bytes from the specified byte array to this output stream. The general contract for `write(b)` is that it should have exactly the same effect as the call `write(b, 0, b.length)`.

Parameters:

`b - the data.`

Throws:

`IOException - if an I/O error occurs.`

See Also:

`write(byte[], int, int)`

write

```
public void write(byte[] b,
                  int off,
                  int len)
           throws IOException
```

Writes `len` bytes from the specified byte array starting at offset `off` to this output stream. The general contract for `write(b, off, len)` is that some of the bytes in the array `b` are written to the output stream in order; element `b[off]` is the first byte written and `b[off+len-1]` is the last byte written by this operation.

The `write` method of `OutputStream` calls the write method of one argument on each of the bytes to be written out. Subclasses are encouraged to override this method and provide a more efficient implementation.

If `b` is `null`, a `NullPointerException` is thrown.

If `off` is negative, or `len` is negative, or `off+len` is greater than the length of the array `b`, then an `IndexOutOfBoundsException` is thrown.

Parameters:

`b - the data.`

`off - the start offset in the data.`

`len - the number of bytes to write.`

Throws:

`IOException - if an I/O error occurs. In particular, an IOException is thrown if the output stream is closed.`

flush

```
public void flush()
           throws IOException
```

Flushes this output stream and forces any buffered output bytes to be written out. The general contract of `flush` is that calling it is an indication that, if any bytes previously written have been buffered by the implementation of the output stream, such bytes should immediately be written to their intended destination.

If the intended destination of this stream is an abstraction provided by the underlying operating system, for example a file, then flushing the stream guarantees only that bytes previously written to the stream are passed to the operating system for writing; it does not guarantee that they are actually written to a physical device such as a disk drive.

The `flush` method of `OutputStream` does nothing.

Specified by:

`flush in interface Flushable`

Throws:

`IOException - if an I/O error occurs.`

close

```
public void close()
           throws IOException
```

Closes this output stream and releases any system resources associated with this stream. The general contract of `close` is that it closes the output stream. A closed stream cannot perform output operations and cannot be reopened.

The `close` method of `OutputStream` does nothing.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Throws:

`IOException` - if an I/O error occurs.

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.